

Cómo Resolver Problemas

Pensamiento Lógico antes que Código

Clase de Programación

Profesor: Mateo Taito Stambuk

Email: mateo.taitos@edu.uai.cl

3 de Junio, 2026

layout: section

0

Nueva Tarea - Entrega 10/06

Nueva Tarea

Simulador de Banco - Registro de Transacciones

Enunciado:

Usted es funcionario de banco. Se maneja un registro histórico de transacciones sobre las cuentas de los clientes.

Requisitos:

- Simular datos históricos con 6 clientes imaginarios
- Cada cliente debe tener entre 10 y 20 transacciones aleatorias (ingresos o egresos)
- Preguntar el nombre del cliente y mostrar su saldo disponible
- El saldo puede ser positivo o negativo

Nueva Tarea

Ejemplo de ejecución esperada

Ingrese nombre del cliente: Juan Pérez

Transacciones de Juan Pérez:

Depósito: +\$150.000

Giro: -\$50.000

Depósito: +\$80.000

Transferencia: -\$30.000

...

Saldo disponible: \$250.000

Fecha de entrega: 10 de Junio, 2026

Objetivos de la Clase

- **Comprender** la importancia del pensamiento lógico
- **Aprender** a leer y separar un problema en partes
- **Utilizar** diagramas de flujo como herramienta de planificación
- **Practicar** el ciclo: planificar, implementar, revisar

1 El Problema del Día

El Problema

Encontrar el Mayor de Tres Números

Enunciado:

Dado tres números ingresados por el usuario, determinar cuál es el mayor y mostrarlo en pantalla.

Pregunta clave: ¿Cómo lo resolvemos?

- ¿Escribimos código directamente?
- ¿O pensamos primero?

2

Lógica vs Código

Lógica vs Codificación

Dos habilidades diferentes

Lógica

- ¿Qué necesito resolver?
- ¿Qué pasos debo seguir?
- ¿En qué orden?
- Se piensa, no se escribe

Codificación

- Traducir la lógica a código
- Usar la sintaxis correcta
- Manejar detalles técnicos
- Se escribe después de pensar

Primero la lógica, después el código

¿Por qué pensar antes de escribir?

- Evita errores lógicos: el código puede compilar pero dar resultados incorrectos
- Ahorra tiempo: es más fácil corregir un plan que reescribir código
- Mejora la claridad: sabes exactamente qué hace cada parte
- Facilita la comunicación: puedes explicar tu solución a otros

Analogía: No construyes una casa sin un plano. Tampoco debes escribir código sin un plan.

3

Diagramas de Flujo

Diagramas de Flujo

Herramienta de planificación visual

¿Qué es?

Un diagrama que muestra los pasos de un proceso usando figuras geométricas.

¿Para qué sirve?

- Visualizar el flujo lógico
- Identificar decisiones y caminos
- Comunicar la solución a otros

Figuras básicas:

Figura	Significado
Óvalo	Inicio / Fin
Rectángulo	Proceso / Acción
Rombo	Decisión (Sí/No)
Paralelogramo	Entrada / Salida

Ejemplo de Diagrama

Flujo simple: ¿Es par o impar?

4

Estructura Típica

Estructura de Resolución

Tres bloques fundamentales

1. Inicialización

Definir variables y valores
iniciales

```
a = 0  
b = 0  
c = 0  
mayor = 0
```

2. Lógica

Procesar datos y tomar
decisiones

```
Si a > b y a > c:  
    mayor = a  
Sino si b > c:  
    mayor = b  
Sino:  
    mayor = c
```

3. Salidas

Mostrar resultados al
usuario

```
Mostrar "El mayor es: "  
Mostrar mayor
```

Esta estructura aplica a la mayoría de los problemas simples

5

Leer el Problema

Cómo Leer un Problema

Separar en partes

Problema: Dado tres números ingresados por el usuario, determinar cuál es el mayor y mostrarlo en pantalla.

1. Variables (¿Qué datos tengo?)

- Tres números: `a` , `b` , `c`
- Un resultado: `mayor`

Cómo Leer un Problema

Separar en partes (continuación)

2. Lógica (¿Qué debo hacer?)

- Comparar los tres números
- Encontrar cuál es el mayor

3. Resultados esperados (¿Qué debo mostrar?)

- El número mayor

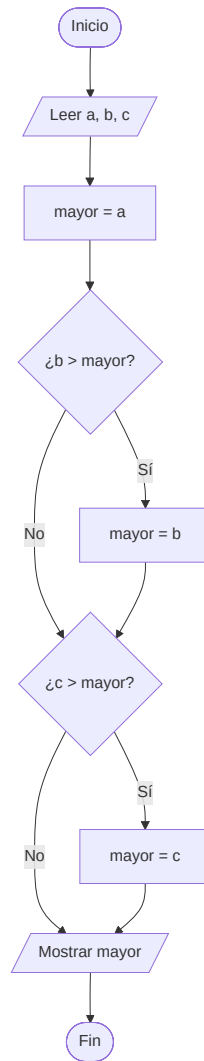
6

Diagrama de Flujo

Diagrama de Flujo

Solución al problema del mayor

Lógica clara antes de escribir código



7

Llevar a Código

De Diagrama a Código

Paso a paso

```
# 1. Inicialización
a = int(input("Ingrese primer número: "))
b = int(input("Ingrese segundo número: "))
c = int(input("Ingrese tercer número: "))

# 2. Lógica
mayor = a
if b > mayor:
    mayor = b
if c > mayor:
    mayor = c

# 3. Salida
print(f"El mayor es: {mayor}")
```

- Línea 1-2: Inicializamos las variables de entrada
- Línea 4-5: Asumimos que `a` es el mayor
- Línea 7-12: Comparamos con `b` y `c`, actualizando si encontramos uno mayor
- Línea 14: Mostramos el resultado

Comparación: Diagrama vs Código

Diagrama:

Código:

```
a = int(input("Número 1: "))
b = int(input("Número 2: "))
c = int(input("Número 3: "))

mayor = a
if b > mayor:
    mayor = b
if c > mayor:
    mayor = c

print(f"Mayor: {mayor}")
```

El código es una traducción directa del diagrama

8

Revisar la Solución

Revisar la Solución

Debugging manual: seguir las variables

Caso de prueba: $a = 5$, $b = 3$, $c = 8$

Paso	Código ejecutado	Estado de variables
1	<code>a = 5, b = 3, c = 8</code>	$a=5, b=3, c=8$
2	<code>mayor = a</code>	$\text{mayor}=5$
3	<code>¿b > mayor?</code> → <code>¿3 > 5?</code>	No, no cambia
4	<code>¿c > mayor?</code> → <code>¿8 > 5?</code>	Sí, actualiza
5	<code>mayor = c</code>	$\text{mayor}=8$

Otro Caso de Prueba

Verificar restricciones

Caso con números iguales: $a = 5$, $b = 5$, $c = 5$

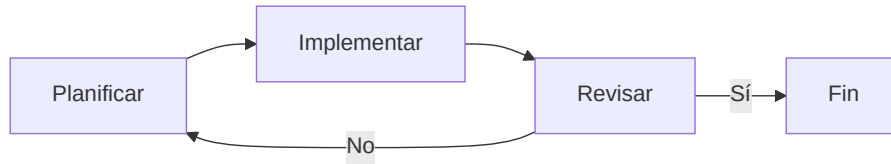
Paso	Código ejecutado	Estado de variables
1	<code>a = 5, b = 5, c = 5</code>	$a=5, b=5, c=5$
2	<code>mayor = a</code>	$mayor=5$
3	<code>¿b > mayor? → ¿5 > 5?</code>	No (no es mayor)
4	<code>¿c > mayor? → ¿5 > 5?</code>	No (no es mayor)
5	<code>print(mayor)</code>	Muestra: 5

9

El Ciclo Completo

Ciclo de Resolución

Planificar → Implementar → Revisar



Planificar

- Leer el problema completo
- Identificar variables, lógica, salidas, restricciones
- Dibujar diagrama de flujo

Implementar

- Traducir el diagrama a código
- Seguir la estructura: Inicialización → Lógica → Salidas

10

Lo que Aprendimos

Resumen

Conceptos clave de la clase

- Pensar antes de escribir: la lógica va antes que el código
- Separar el problema: variables, lógica, salidas, restricciones
- Diagramas de flujo: herramienta visual para planificar
- Estructura básica: inicialización → lógica → salidas
- Revisar la solución: probar con casos de prueba manualmente
- Ciclo completo: planificar → implementar → revisar

Regla de oro: Un buen programador piensa más de lo que escribe.

11

Desafío

Desafío (15 minutos)

Suma de Matrices 3x3

Problema:

Dadas dos matrices de 3x3 ingresadas por el usuario (usando listas anidadas y ciclos `for`), calcular la suma de ambas matrices y mostrar el resultado.

Ejemplo:

Matriz A:	Matriz B:	Resultado:
1 2 3	4 5 6	5 7 9
4 5 6	7 8 9	11 13 15
7 8 9	1 2 3	8 10 12

Pasos sugeridos:

¡Gracias!
¿Preguntas?

Cómo Resolver Problemas: Pensamiento Lógico